

Interblocage

Introduction

L'interblocage peut être défini comme un blocage permanent d'un ensemble de processus concurrents (processus en compétition pour acquérir des ressources du système matérielles ou logicielles). Un ensemble de processus est en interblocage si chaque processus dans l'ensemble attend un événement que seul un autre processus peut provoquer.

L'interblocage est permanent parce qu'aucun de ces événements ne peut être déclenché.

En général, il n'y a aucune solution effective pour le problème d'interblocage.

Catégories de ressources :

Ressources réutilisables : c'est une ressource qui peut être utilisée sans risque par un seul processus à la fois et ne sera pas épuisée par cet usage. Les processus obtiennent des unités de ressources qui les libèrent ultérieurement pour les réutiliser par d'autres processus.

Exemple : les processeurs, canaux E/S, lecteur CD, mémoire principale, les BDD, les fichiers.

Ressources consommables : c'est une ressource qui peut être créée (produite) et peut être détruite (consommée). Quand une ressource est acquise par un processus consommateur, cette ressource cesse d'exister.

Exemple : interruptions, signaux, messages, info sur les tampons.

Conditions d'interblocage :

Trois conditions doivent être présentes pour qu'un interblocage :

- *Exclusion Mutuelle* : chaque ressource est soit attribuée à un seul processus, soit disponible.
- *Détention et attente* : les processus ayant déjà obtenu des ressources peuvent en demander de nouvelles.
- *Pas de réquisition* : aucune ressource ne peut être retirée de force à un processus qui la détient (libération explicite).

Les trois premières conditions sont nécessaires mais pas suffisantes pour engendrer un interblocage. Pour qu'un interblocage se produise réellement, une quatrième condition est exigée (qui est une conséquence potentielle des trois premières).

- *Attente circulaire* : l'existence d'un cycle fermé d'au moins deux (2) processus, chacun attendant une ressource détenue par un autre processus du cycle.

Possibilité d'interblocage	Existence d'interblocage
1. <i>Exclusion Mutuelle</i>	1. <i>Exclusion Mutuelle</i>
2. <i>Détention et attente</i>	2. <i>Détention et attente</i>
3. <i>Pas de réquisition</i>	3. <i>Pas de réquisition</i>
	4. <i>Attente circulaire</i>

Graphe d'allocation des ressources

C'est un outil utile pour caractériser l'attribution des ressources aux processus. C'est un graphe orienté qui représente l'état des ressources et des processus dans un système. Chaque processus et chaque ressource est représenté par un nœud.

Une flèche (arc) du graphe dirigée d'un processus vers une ressource indique que le processus demande la ressource qui ne lui est pas encore accordée. Les points dans un nœud d'une ressource, indiquent le nombre d'instances de cette ressource. Une flèche (arc) dirigée d'un point d'une ressource réutilisable vers un processus indique qu'une unité de cette ressource est détenue par le processus. Une flèche dirigée d'un point d'une ressource consommable vers un processus indique que ce processus est le producteur de cette ressource.

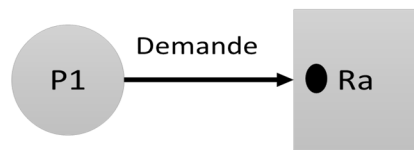
On peut décrire l'état d'allocation des ressources d'un système en utilisant un graphe.

- Le graphe **G** est constitué d'un ensemble de sommets **V** et d'arcs **E** :
- **V** est composé de :
 - $P = \{P_1, P_2, \dots, P_n\}$ l'ensemble de tous les processus dans le système.
 - $R = \{R_1, R_2, \dots, R_m\}$ l'ensemble de tous les types de ressources dans le système
- **E** peut être :
 - Arc de requête ou demande $P_i \rightarrow R_j$
 - Arc d'allocation $R_i \rightarrow P_j$

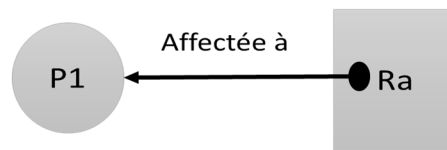
Quand un processus **P_i** demande une instance du type de ressource **R_j**, un arc de requête (demande) est inséré dans le graphe d'allocation des ressources. Quand cette requête (demande) peut être satisfaite, l'arc de requête (demande) est instantanément transformé en un arc d'affectation. Quand plus tard, le processus libère la ressource, l'arc d'affectation est supprimé.

- On désigne un processus avec la notation
- et la ressource avec



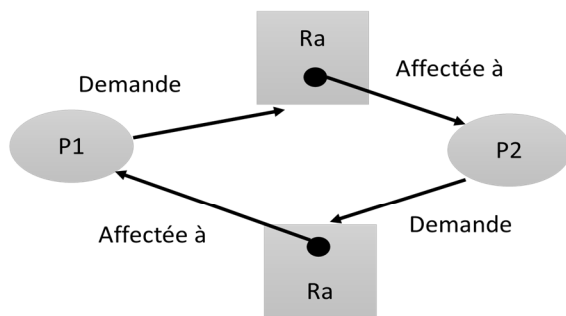


(a) Ressource demandée

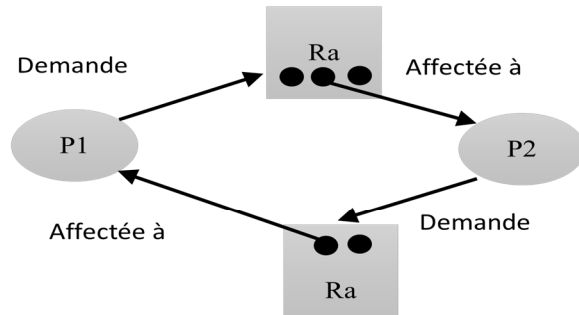


(b) Ressource affectée

Exemple :



Attente circulaire



Pas d'interblocage

Traitement des interblocages

1. **Ignorance des problèmes** : l'approche la plus simple est la politique de l'autruche. La plupart des systèmes d'exploitation se contentent de d'ignorer le problème, la plus part des utilisateurs préfèrent être confrontés à un interblocage plutôt que d'être soumis à une règle qui limiterait tous les utilisateurs à un seul processus ou un seul fichier ouvert.
2. **Prévention d'interblocage** : son principe consiste à attaquer aux conditions de l'interblocage.

Conditions	approche
Exclusion Mutuelle	Tout traiter en différé (spooling)
Détention et attente	Demander toutes les ressources dès le départ
Pas de réquisition	Retirer des ressources
Attente circulaire	Ordonner les ressources numériquement

3. **Détection des interblocages** : le système ne cherche pas à, empêcher les interblocages, il les laisse se produire, et il essaie de les détecter et d'y remédier ultérieurement (préemption, point de reprise, suspension d'un processus).

On dit que l'état du système est interbloqué si dans la progression des exécutions des processus on passe par plusieurs états $\text{état}_1 \rightarrow \text{état}_2 \rightarrow \dots \rightarrow \text{état}_n$ où après le dernier état on ne peut pas passer vers un autre alors que cet état contient des processus qui ne peuvent pas se progresser dans leurs exécutions.

On peut suivre ces états par la réduction du graphe de processus-ressource ou la formalisation mathématique de l'interblocage.

3.1 Algorithme de Parcours en Profondeur (DFS: Deep First Search)

1. Initialiser L une pile vide et designer tous les arcs comme non marqué
2. Pour chaque nœud n :
3. Si le nœud n n'est pas dans la pile L , l'ajouter à L
Sinon, il existe un cycle et l'algorithme prend fin
4. S'il n'existe pas un arc non marqué sortant de n , dépiler n

Si la pile est vide, aller à l'étape 2

Sinon le nœud au sommet de la pile devient n et revenir à l'étape 4

Sinon choisir au hasard un arc sortant non marqué et marquer le, pointer sur le nœud au bout de l'arc choisi qui devient n et revenir à l'étape 3

Si l'algorithme ne se termine pas à l'étape 3, il n'y a pas de cycle (pas d'interblocage)

3.2 Réduction de graphe

On dit qu'un graphe est réduit par un processus P_i qui n'est pas bloqué et qui n'est pas isolé si ce processus peut acquérir toutes les ressources puis les libérer toutes (suppression de tous les arcs allant de et vers P_i).

Cette méthode est utilisée pour la détection de l'interblocage dans le cas des ressources à un seul exemplaire.

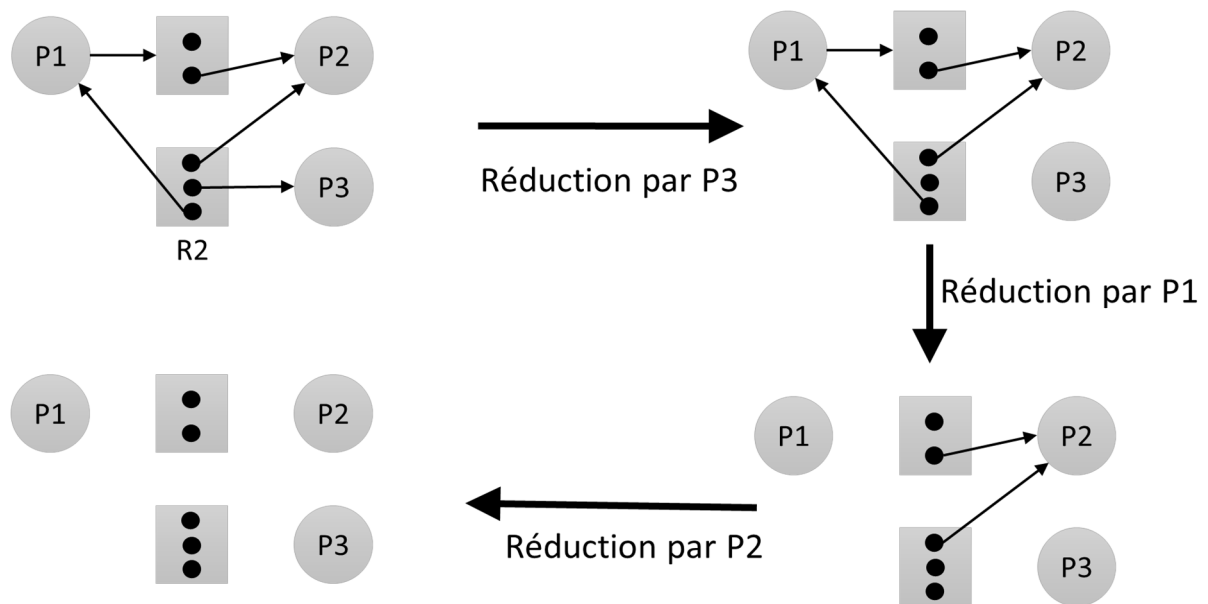
- Un graphe est dit **irréductible** s'il ne peut être réduit dans un état n d'évolution des processus ainsi la présence d'un cycle.
- Un graphe est **réductible** s'il existe une séquence d'état dont le dernier état ne laisse aucun arc.

3.3 Algorithme de réduction de graphe

- Si une ressource possède seulement des flèches qui sortent (il n'y a pas des requêtes), on les efface.
- Si un processus possède seulement des flèches qui pointent vers lui, on les efface.

- Si une ressource a des flèches qui sortent, mais pour chaque flèche de requête il y a une ressource disponible dans le bloc de ressources où la flèche pointe, alors ses flèches de requêtes **sont transformées** en flèches d'allocations.
- La procédure est répétée jusqu'à impossibilité de réduction
- Le système n'est pas en interblocage s'il ne reste plus aucun arc dans le graphe.

Exemple



4. Évitement des interblocages :

Vecteurs et état :

- Matrice des demandes (**R**) : est un vecteur qui contient toutes les demandes (toujours en attente) des ressources formulées par les processus.
- Matrice des allocations (**A**) : est un vecteur qui contient les allocations courantes des ressources aux processus (déjà attribuées).
- Vecteur des ressources disponible (**D**) : est un vecteur qui contient les ressources non encore attribuées.

L'algorithme du banquier

- 1- Rechercher une rangée dans R dont les demandes sont $\leq D$. s'il n'en existe pas alors fin de l'algorithme (interblocage)
- 2- Supposons que le processus de la rangée choisie demande toutes les ressources nécessaires et qu'il se termine, alors marqué ce processus comme achevé et ajouter toutes ces ressources au vecteur D.
- 3- Recommencer 1 et 2. On aura deux possibilités :
 - Tous les processus seront marqués comme achevés (état initiale sure)
 - Interblocage (état initial non sure)

Exercice

Soient 4 processus (P1, P2, P3 et P4) qui s'exécutent simultanément sur un système qui possède 4 types de ressources (R1, R2, R3 et R4). Supposons que le système est à l'état suivant :

R : Demandes				
	R1	R2	R3	R4
P1	0	1	0	2
P2	0	0	1	0
P3	0	0	0	1
P4	1	0	0	1

A : Allocations				
	R1	R2	R3	R4
P1	1	0	1	1
P2	1	1	0	0
P3	1	0	1	1
P4	0	0	1	0

D : Disponibles			
R1	R2	R3	R4
0	0	0	1

- 1- Le système est-il en état sûr ? justifier votre réponse.
- 2- Supposons qu'à partir de cet état le processus **P3** libère une unité de **R1 et R3** et **P2** libère une unité de **R2**, et **P1** libère une unité de **R4**.
 - a) Ce nouvel état est-il sûr ?
 - b) À partir de ce nouvel état, le processus **P2** fait une demande de ressources **(1, 1, 0,1)**. Est-ce que le système peut accorder cette demande.

Solution de l'exercice

1) Pour vérifier l'état de ce système, il faut appliquer l'algorithme du banquier sur son état actuel :

- 1- Demande de P3 $\leq$ D, Marquer P3, $D = D + \text{Allocation de P3}$, $D = (1, 0, 1, 2)$
- 2- Demande de P2 $\leq$ D, Marquer P2, $D = D + \text{Allocation de P2}$, $D = (2, 1, 1, 2)$
- 3- Demande de P1 $\leq$ D, Marquer P1, $D = D + \text{Allocation de P1}$, $D = (3, 1, 2, 3)$
- 4- Demande de P4 $\leq$ D, Marquer P4, $D = D + \text{Allocation de P4}$, $D = (3, 1, 3, 3)$

Tous les processus sont marqués, alors l'état est sûr.

2) Le nouvel état sera :

R : Demandes				
	R1	R2	R3	R4
P1	0	1	0	3
P2	0	1	1	0
P3	1	0	1	1
P4	1	0	0	1

A : Allocations				
	R1	R2	R3	R4
P1	1	0	1	0
P2	1	0	0	0
P3	0	0	0	1
P4	0	0	1	0

D : Disponibles			
R1	R2	R3	R4
1	1	1	2

a) Vérification de l'état

- 1- Demande de P2 $\leq$ D, Marquer P2, $D = D + \text{Allocation de P2}$, $D = (2, 1, 1, 2)$
 - 2- Demande de P3 $\leq$ D, Marquer P3, $D = D + \text{Allocation de P3}$, $D = (2, 1, 1, 3)$
 - 3- Demande de P1 $\leq$ D, Marquer P1, $D = D + \text{Allocation de P1}$, $D = (3, 1, 2, 3)$
 - 4- Demande de P4 $\leq$ D, Marquer P4, $D = D + \text{Allocation de P4}$, $D = (3, 1, 3, 3)$
- Le nouvel état est un état sûr. Car tous les processus sont marqués.

b) Traitement de la demande (0, 1, 0, 0) du processus P2

On suppose que la demande est accordée, et on applique l'algorithme du banquier pour vérifier ce nouvel état.

R : Demandes				
	R1	R2	R3	R4
P1	0	1	0	3
P2	0	2	1	0
P3	1	0	1	1
P4	1	0	0	1

A : Allocations				
	R1	R2	R3	R4
P1	1	0	1	0
P2	1	1	0	0
P3	0	0	0	1
P4	0	0	1	0

D : Disponibles			
R1	R2	R3	R4
1	0	1	2

1- Demande de P3 $\leq$ D, Marquer P3, $D = D + \text{Allocation de P3}$, $D = (1, 0, 1, 3)$

2- Demande de P4 $\leq$ D, Marquer P4, $D = D + \text{Allocation de P4}$, $D = (1, 0, 2, 3)$

Les demandes des processus P1 et P2 ne peuvent pas être satisfaites. Par conséquent, P1 et P2 ne peuvent pas être exécutés. Ce nouvel état est un état non sûr (deadlock). Donc, pour éviter cet interblocage, le système ne doit pas accorder la demande de P2.